

ML: страх и ненависть в продакшне

Паша Финкельштейн, JetBrains

Михаил Марюфич, [OK.ru](https://ok.ru)

О себе

- 14 лет в IT
- 11 в разработке с разных концов
- 2 года в DE
- Из них год в [Big Data Tools](#) *

**Big Data Tools – инструмент для упрощения жизни дата инженеров*

Михаил Марюфич

- Закончил Мат-Мех СПбГУ и Computer Science Center
- Machine Learning Engineer в [OK.ru](https://ok.ru)
- Делаю МЛ штуки и внедряю их в продакшен
- Увлекаюсь воспроизводимостью



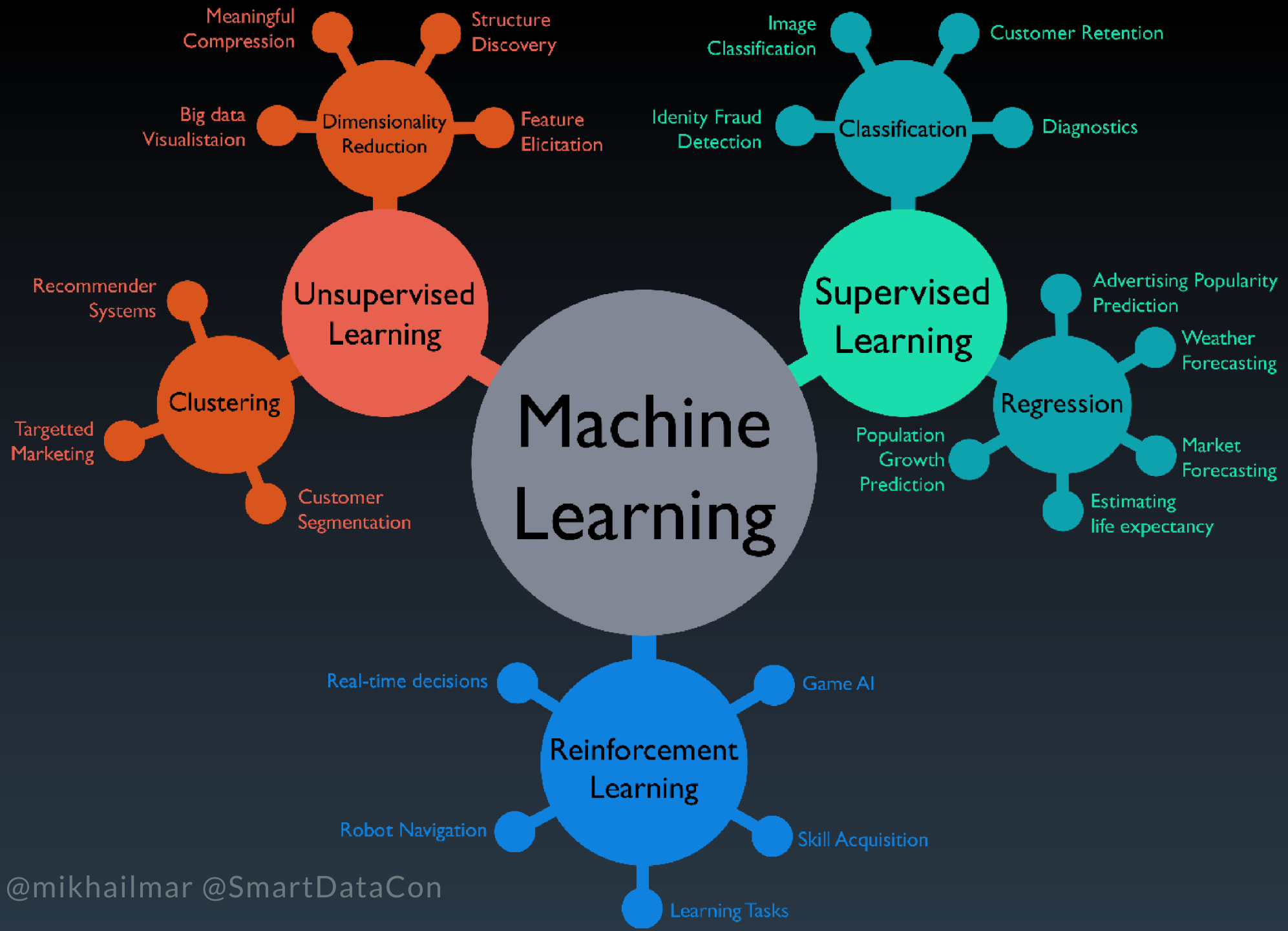
Это доклад о том, что такое MLOps, зачем он нужен и как можно попробовать его делать

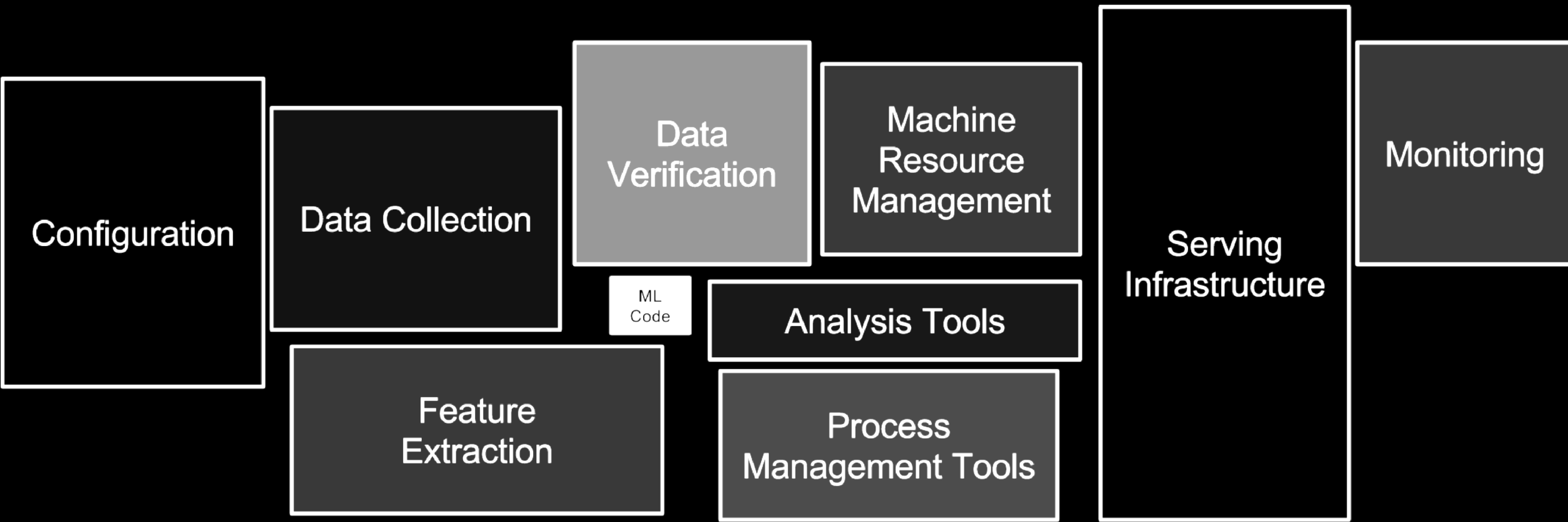
Концепции общие, инструменты меняются

О древнейшей истории

- Буча началась с гиганта
- И гигантом был Google

<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>



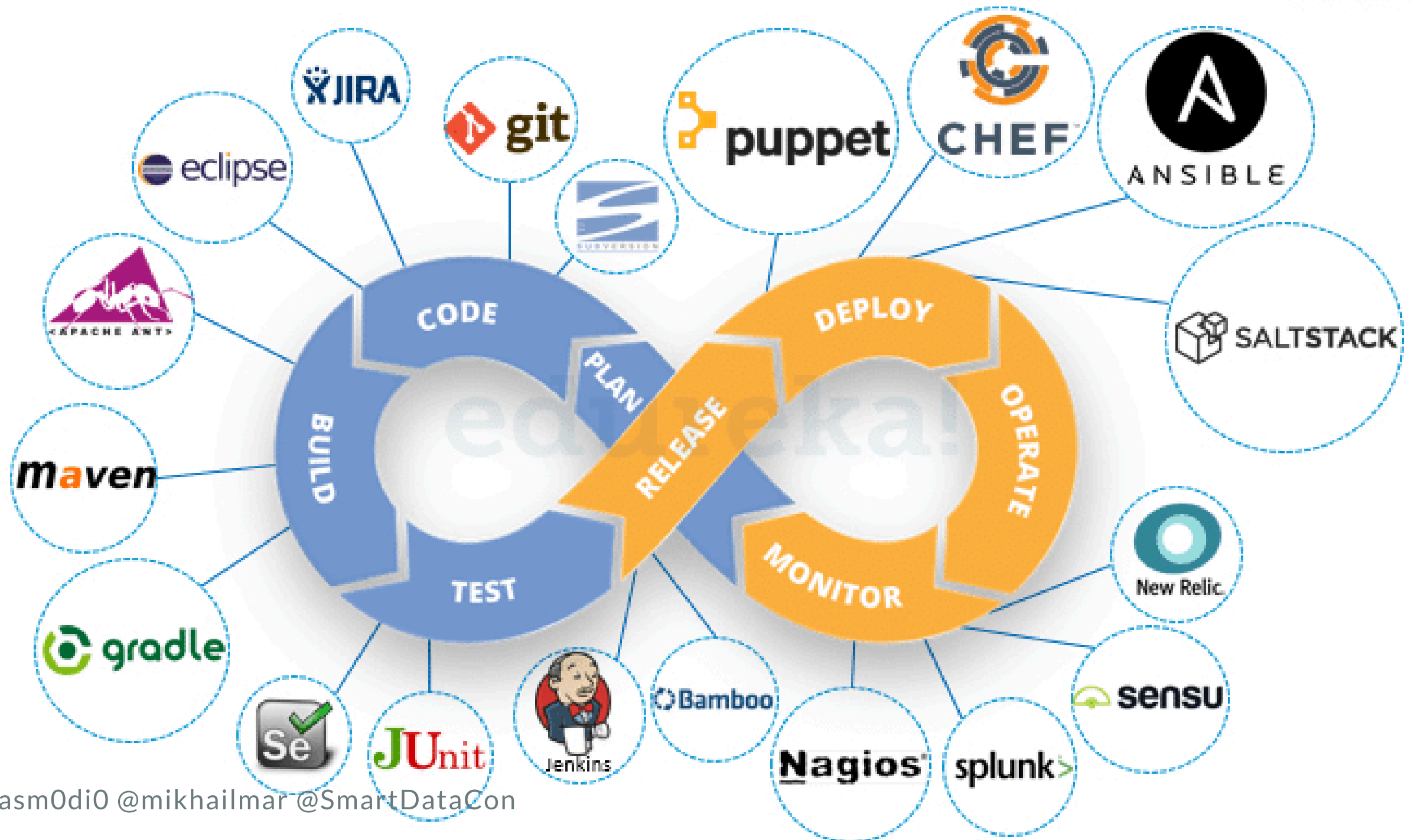


“**Мы все неправы!**

”

Google

Потому что не продуктивизируем свой ML код.



DevOps для обычной разработки

Тем временем в обычной разработке

- Инструментов тонна
- Все знают, как продуктивизировать разработку
- Все понимают, где светлое будущее

И конечно же вокруг этого навёрнута тонна практик

Operations Anti-Patterns, DevOps Solutions

Jeffery D. Smith

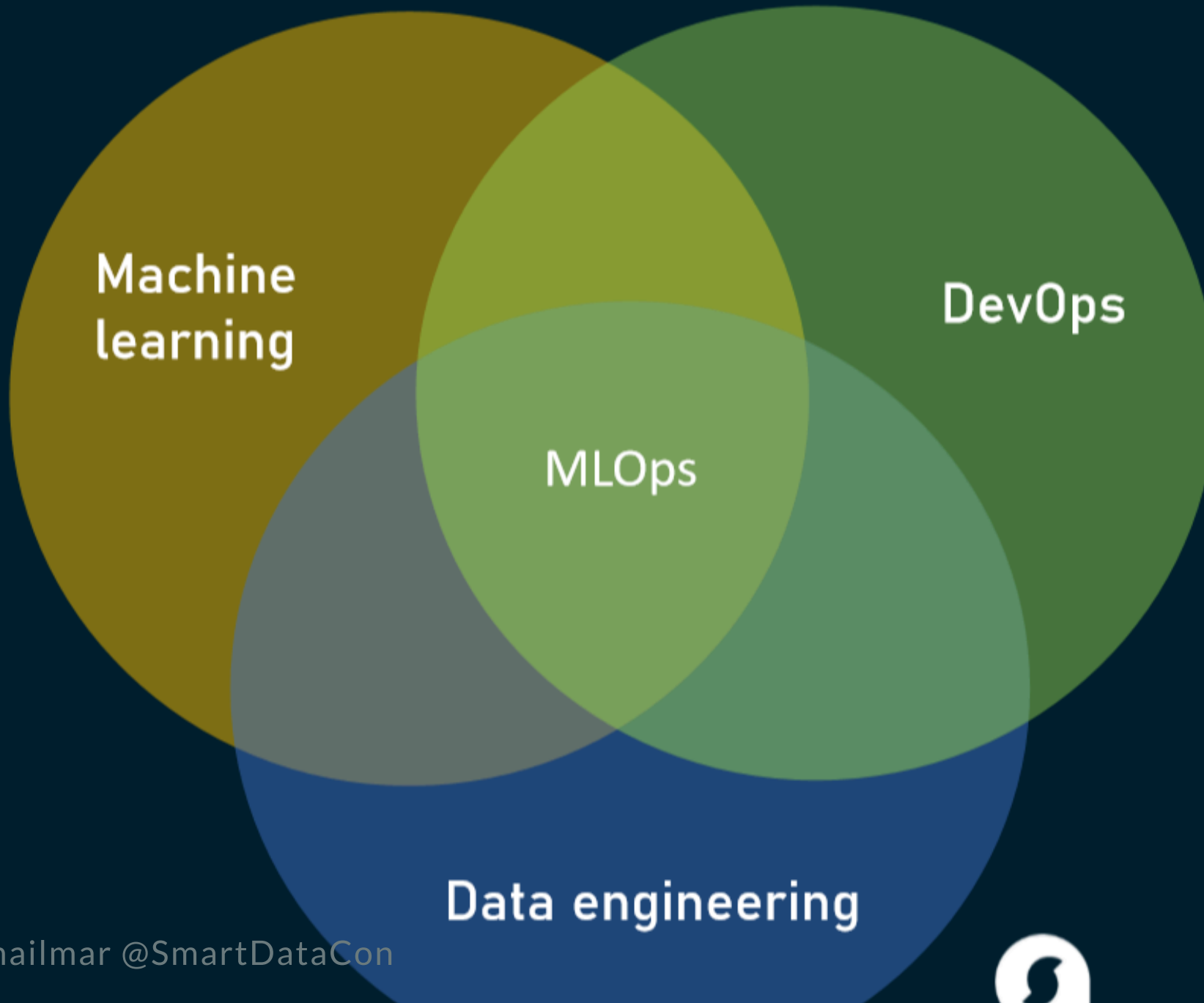


“ DevOps isn’t about tools, but about how teams work together ”

**Новая книга про
анти-паттерны Ops и
решения DevOps**

Так что, MLOps — это просто DevOps?

Нет!



**MLOps — это ещё одна ступень
специализации**

Как устроен ML



Data

Schema

Sampling over Time

Volume



Model

Algorithms

More Training

Experiments



Code

Business Needs

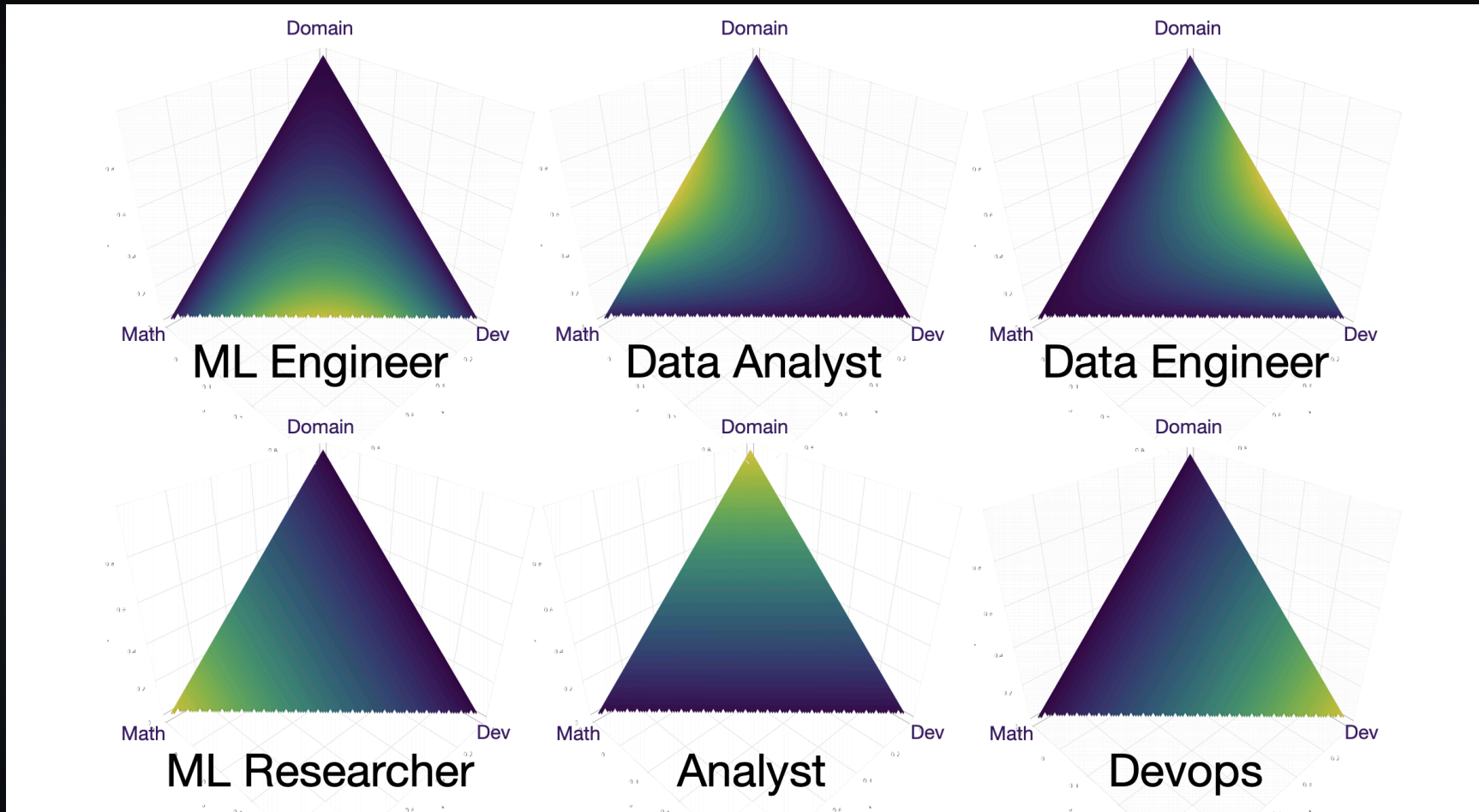
Bug Fixes

Configuration

А кто такие дата саентисты?

- Data — данные, тут всё понятно.
 - копаются в реке и находят золото
- Scientists — учёные. Экспериментаторы

Роли в ML разработке



Требования к эксперименту

- Формулирование гипотезы
- Описание специфических условий
- Воспроизводимость
- Протоколирование

“ MLOps — это способ
сделать эксперименты
научным, а не наколеночным ”

Эйнштейн



Я всегда верю цитатам в интернете

Курт Кобейн

Как устроен ML



Data

Schema

Sampling over Time

Volume



Model

Algorithms

More Training

Experiments



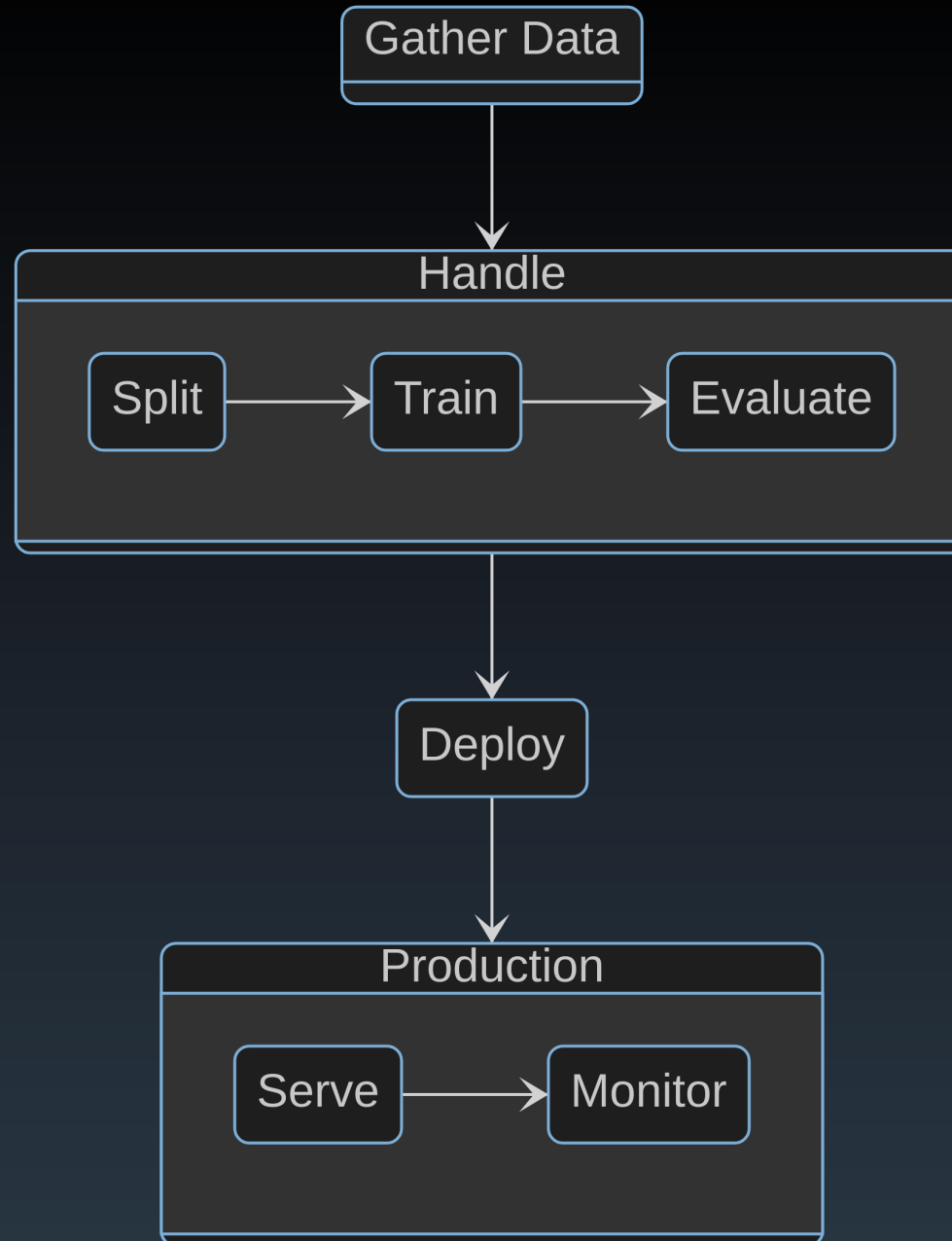
Code

Business Needs

Bug Fixes

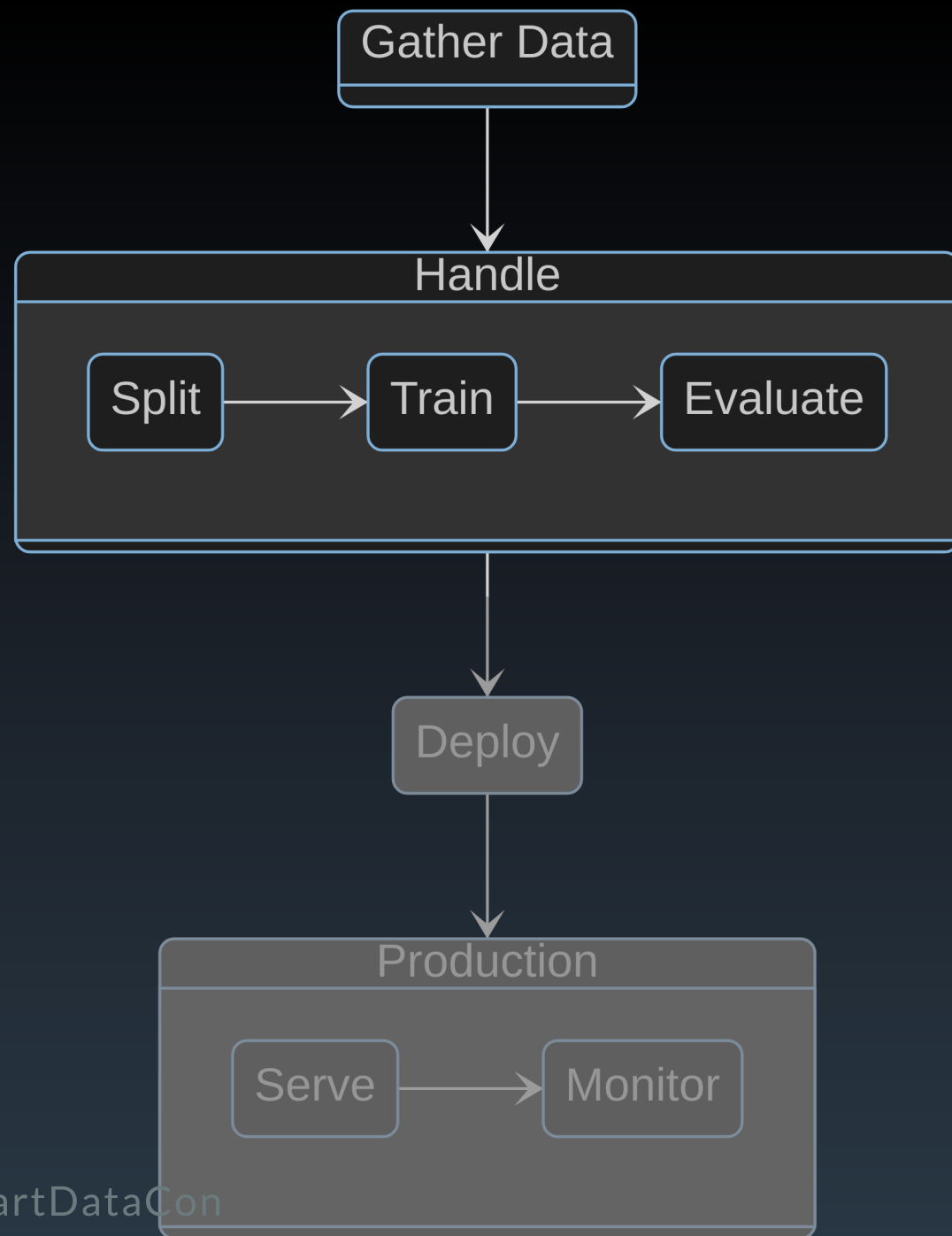
Configuration

ML workflow

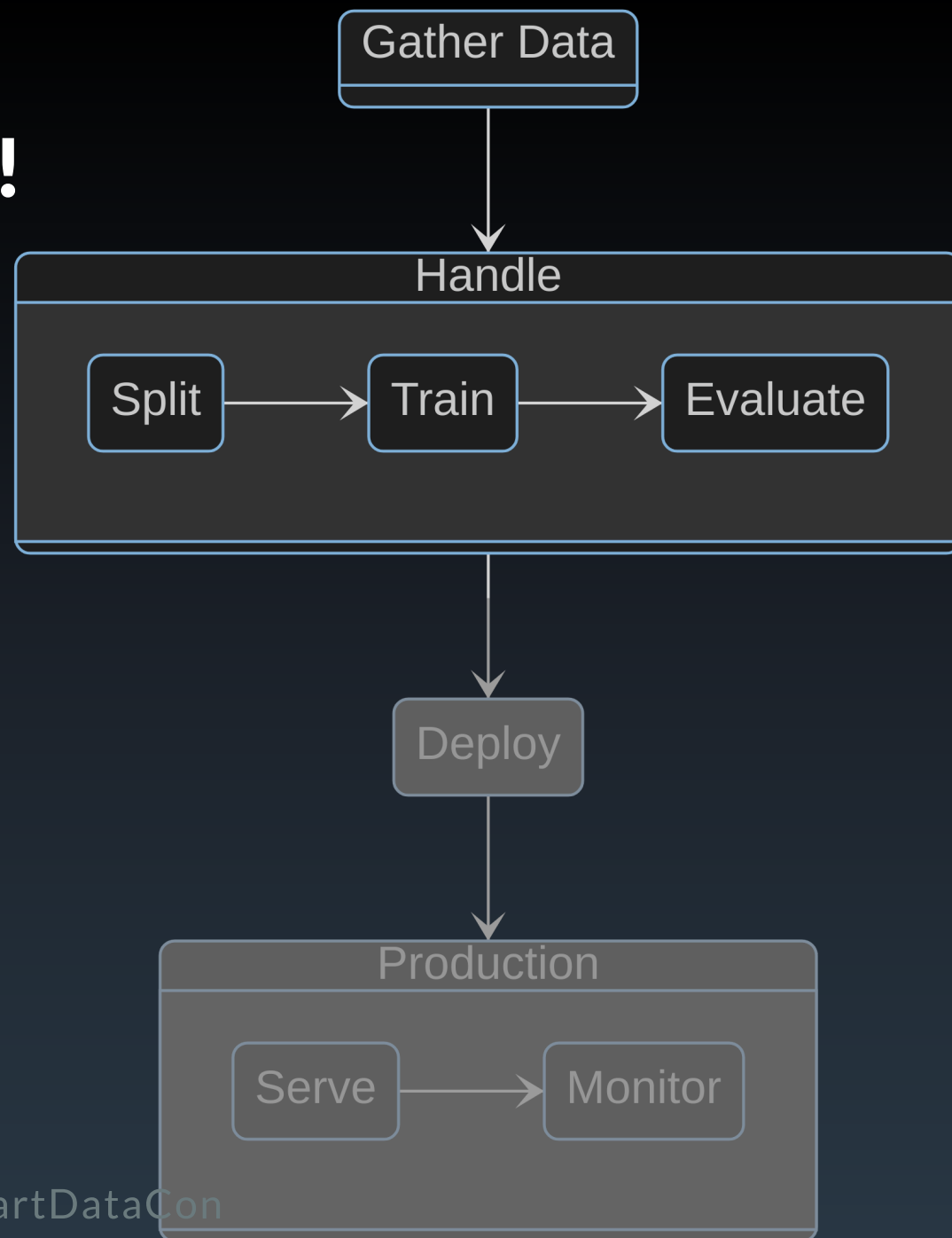


А чем ПРИНЦИПИАЛЬНО отличается от DEVOPS?

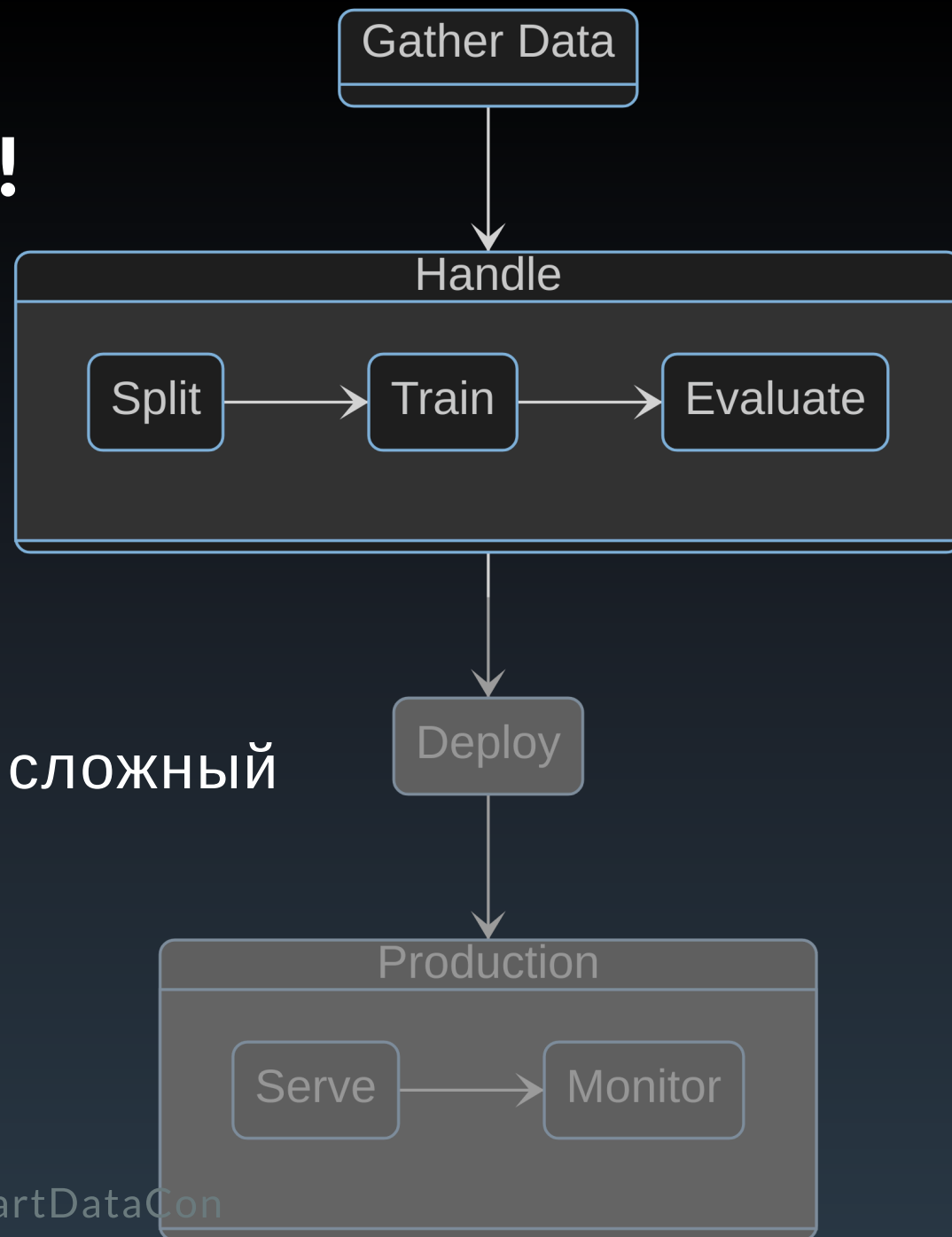




Это же билд!



Это же билд!



PS: только очень сложный

Что тут пошло не так?

- Всё одним куском
- Протокола эксперимента нет
- Информации об исходных данных нет
- Результаты меняются от запуска к запуску

Что делать?

1



download_data.py



Data

2



splitter.py

Training Data



Validation Data



3



evaluation.py



decision_tree.py



model.pkl



metrics.json

Разве это не очевидно?

НЕТ!

Jupyter

```
In [21]: import os
import platform
```

```
In [22]: platform
Out[22]: <module 'platform' from '/opt/anaconda3/lib/python3.6/platform.py'>
```

```
In [23]: from pyspark import SparkConf
from pyspark import SparkContext
```

```
In [24]: sc = SparkContext.getOrCreate()
```

```
In [25]: sc.getConf().getAll()
```

```
Out[25]: [('spark.app.id', 'local-1536486752230'),
('spark.driver.port', '42132'),
('spark.rdd.compress', 'True'),
('spark.serializer.objectStreamReset', '100'),
('spark.master', 'local[*]'),
('spark.executor.id', 'driver'),
('spark.submit.deployMode', 'client'),
('spark.ui.showConsoleProgress', 'true'),
('spark.app.name', 'pyspark-shell'),
('spark.driver.host', 'spark-master.c.mozn-location.internal')]
```

```
In [26]: import numpy as np
TOTAL = 1000000
dots = sc.parallelize([2.0 * np.random.random(2) - 1.0 for i in range(TOTAL)]).cache()
print("Number of random points:", dots.count())
stats = dots.stats()
print('Mean:', stats.mean())
print('stdev:', stats.stdev())
```

```
Number of random points: 1000000
Mean: [0.000752275 0.000510831]
stdev: [0.57730526 0.57748448]
```

```
In [21]: import os
import platform
```

Jupyter

```
In [22]: platform
```

```
Out[22]: <module 'platform' from '/opt/anaconda3/lib/python3.6/platform.py'>
```

```
In [23]: from pyspark import SparkConf
from pyspark import SparkContext
```

```
In [24]: sc = SparkContext.getOrCreate()
```

```
In [25]: sc.getConf().getAll()
```

- Самый популярный (и достаточно удобный) инструмент Data Scientist'a

- Код вперемешку с Markdown

- Написан и выполняется в непредсказуемом порядке

```
Out[25]: [('spark.app.id', 'local-1516416752130'),
('spark.executor.id', 'driver'),
('spark.rdd.compress', 'True'),
('spark.serializer.objectStreamReset', '100'),
('spark.submit.deployMode', 'client'),
('spark.ui.showConsoleProgress', 'true'),
('spark.app.name', 'pyspark-shell'),
('spark.driver.host', 'spark-master.c.mozn-location.internal')]
```

```
In [26]: import numpy as np
TOTAL = 1000000
dots = sc.parallelize([2.0 * np.random.random(2) - 1.0 for i in range(TOTAL)]).cache()
print("Number of random points:", dots.count())
stats = dots.stats()
print('Mean:', stats.mean())
print('stdev:', stats.stdev())
```

```
Number of random points: 1000000
Mean: [0.00375275 0.00510821]
stdev: [0.57730526 0.57748448]
```

Придётся переучивать



DVC

- Создавался специально в пару к гиту
- Изначально был похож на LFS (git-lfs)
- Оброс функциональностью

Примеры

```
git add  
dvc add
```

Добавить какой-то файл под контроль git/dvc

Примеры

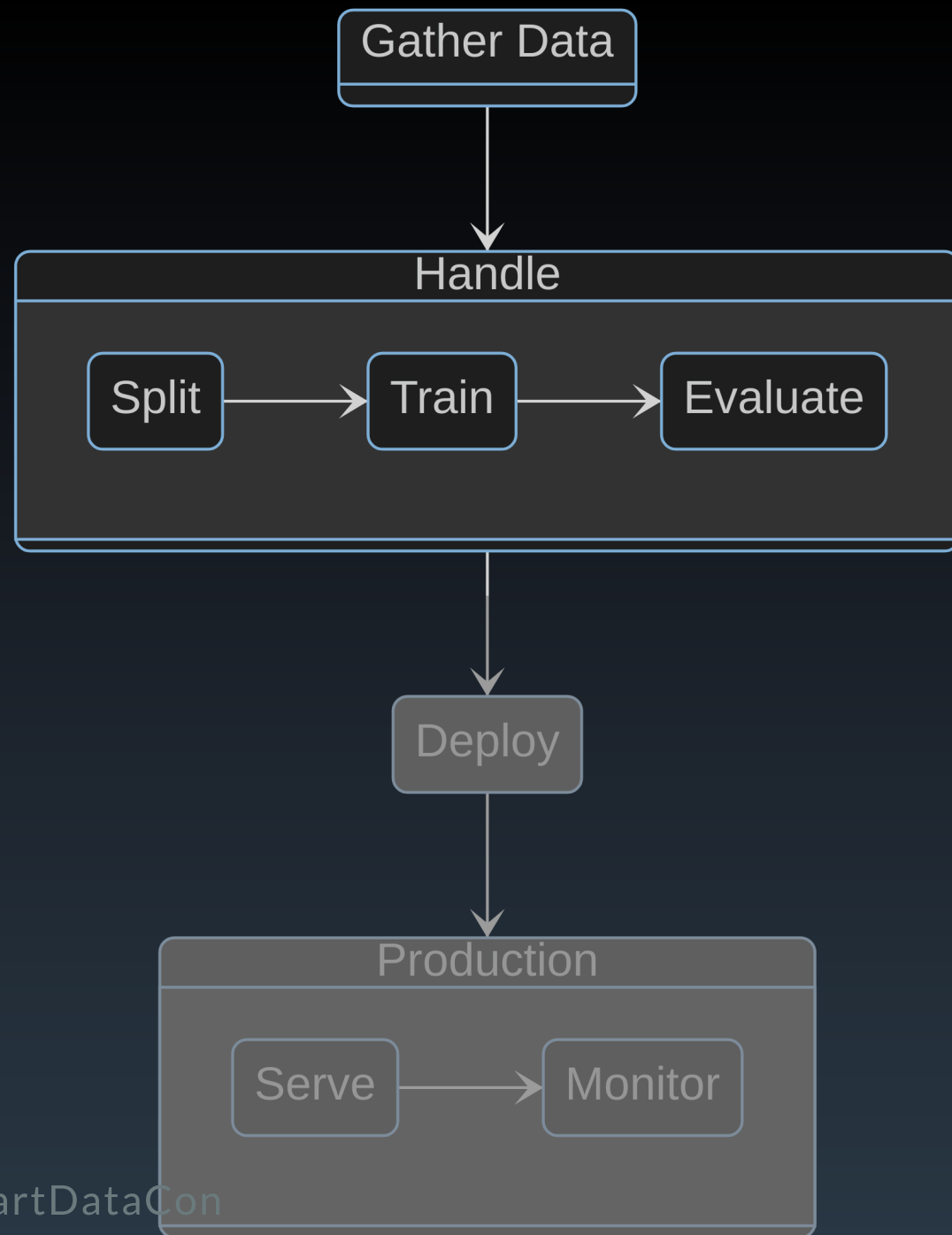
```
git commit  
dvc commit
```

Сохранить состояние файла

Примеры

```
git diff  
dvc diff
```

Посмотреть разницу с каким-то состоянием



The talk is cheap

Show us the demo!

Ок, но ЭТО КОНСОЛЬ

- Молодёжь   не любит терминалы
- Этим неудобно пользоваться при командной разработке
- Тимлиду может быть неудобно работать с метриками

Overview

Diff

Commits

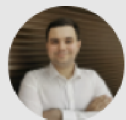
PR - это очень удобно

Details

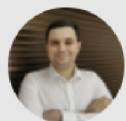


Neural Network Trainer Service created a pull request 9 hours ago
auto request [XPRM-16333] -> [master]

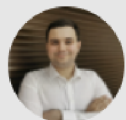
Activity



What do you want to say?



Mikhail Maryufich **MERGED** XPRM-16333 to master in commit [5175918ed7c](#) 8 hours ago



Mikhail Maryufich marked the pull request as **APPROVED** 8 hours ago

Easy



```
mlflow.sklearn.autolog()  
remote_server_uri = "http://127.0.0.1:5000"  
mlflow.set_tracking_uri(remote_server_uri)  
mlflow.set_experiment("udemy-courses")
```

MLFlow Tracking

mlflow

ExperimentsModels

GitHubDocs

Experiments

+<

Search Experiments

Default

udemy-courses

udemy-courses

Experiment ID: 1

Artifact Location: ./mlruns/1

Notes

None

Search Runs: metrics.rmse < 1 and params.model = "tree" and tags.mlflow.source.type = State: Active Search Clear

Showing 2 matching runs Compare Delete Download CSV

						Parameters >	Metrics >	Tags					
☐	Start Time	Run Name	User	Source	Version	bootstrap	ccp_alpha	copy_X	training_r	training_r	training_r	estimator	estimator
☐	2020-11-23 19:1	-	finkel	train_1	9f9980	True	0.0	-	1236.1	9678...	0.897	sklea...	Rand...
☐	2020-11-23 19:1	-	finkel	train_1	479713	-	-	True	3799.9	8854...	0.062	sklea...	Linea...

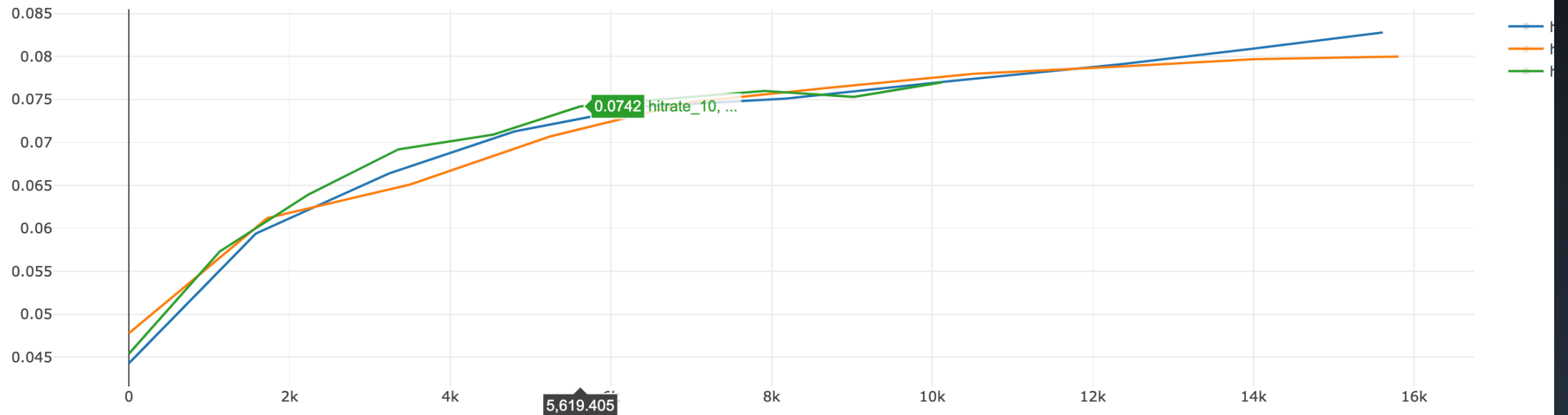
MLFlow Tracking - сравнение запусков

Grid search demo > Comparing 4 Runs

Run ID:	84e2712f7a864c509a52b255394d5e5a	2ff7d4c31f7746b6a18af504e6430014	a8c0994d0d4147e2824e6109d93d2cd5	99ce9f6dad914c3d9a7e744979dd930c
Run Name:	Fork for configuration=42	Fork for configuration=41	Fork for configuration=40	Fork for configuration=39
Start Time:	2019-11-01 17:51:38	2019-11-01 17:51:02	2019-11-01 17:51:01	2019-11-01 17:50:25
Parameters				
elasticNetParam	0.5924460687674582	0.0924460687674582	0.9049460687674582	0.4049460687674582
regParam	0.07031306629069149	0.020313066290691496	0.014063066290691496	0.0640630662906915
Metrics				
auc on test 	0.74	0.744	0.744	0.742
auc on train 	0.736	0.739	0.738	0.736
auc_pr on test 	0.396	0.394	0.398	0.394
auc_pr on train 	0.398	0.398	0.401	0.396

MLFlow Tracking - сравнение запусков

Comparing 3 Runs > hitrate_10



MLFLOW MODEL REGISTRY

Как модель там окажется?

Как модель там окажется?

```
from mlflow.tracking import MlflowClient

client = MlflowClient()
client.create_registered_model("sk-learn-random-forest-reg-model")
```

```
client = MlflowClient()
result = client.create_model_version(
    name="sk-learn-random-forest-reg-model",
    source="mlruns/0/d16076a3ec534311817565e6527539c0/artifacts/sklearn-model",
    run_id="d16076a3ec534311817565e6527539c0"
)
```

Что дальше?

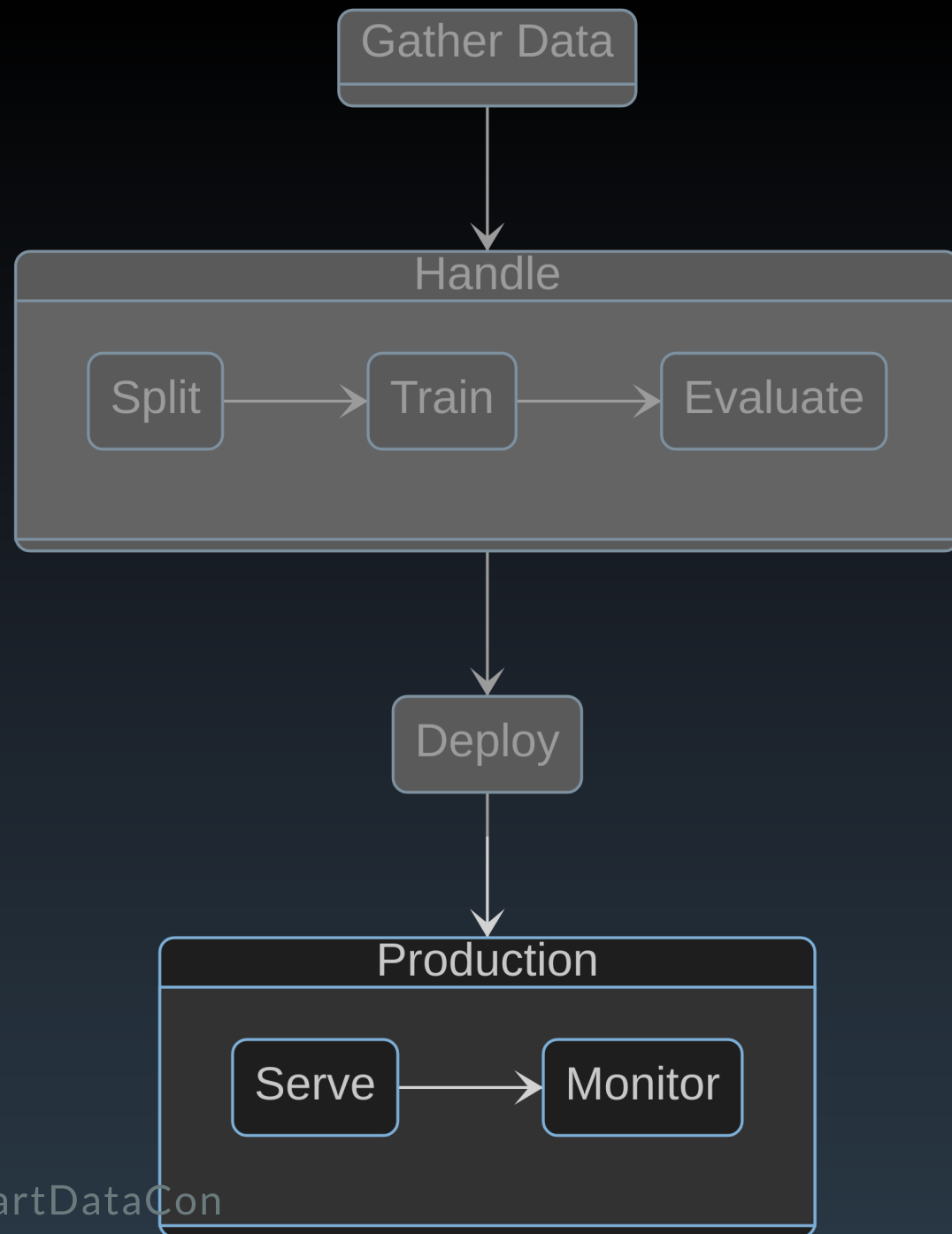
- У нас есть модельки
- У нас есть каталог моделей
- Тестируем!

Тестируем

Мы уже знаем всё, что надо знать про модельки:

1. Среднеквадратичное отклонение
2. Коэффициент детерминации
3. Метрики, специфичные для нашей модели

Всё это можно делать в целом в любом CI!



Serve

- Есть ML MODEL - это бинарник (.pkl, .pth, .h5)
- Нужно делать к ней запросы

Serve

- Есть ML MODEL - это бинарник (.pkl, .pth, .h5)
- Нужно делать к ней запросы
- И каждый стремится написать свой велосипед =(

MLFLOW SERVING



```
$ mlflow models serve -m runs:/my-run-id/model-path &
```

```
$ curl http://1127.0.0.1:5000/invocations -H 'Content-type: application/json' -d '{  
  "columns": ["a", "b", "c", "d"],  
  "data": [[1, 2, 3], [4, 5, 6]]  
}'
```

CORE

1. Containerise
2. Deploy
3. Monitor

From model binary



Or language wrapper



Into fully fledged microservice

Simple Seldon Core Inference Graph

Model A

API
(REST,
gRPC)

Complex Seldon Core Inference Graph

Model B

Model A

Model C

Multi Armed
Bandit

Direct traffic to the
most optimal
model

Feature
Transformation

Outlier
Detection

Key features to
identify outlier
anomalies (*Fraud*,
KYC)

Explanation

Why is the model
doing what it's
doing?

API
(REST,
gRPC)

Show us the demo!

Мониторинг

2 типа инструментов:

1. ML-Специфичные
2. Общие

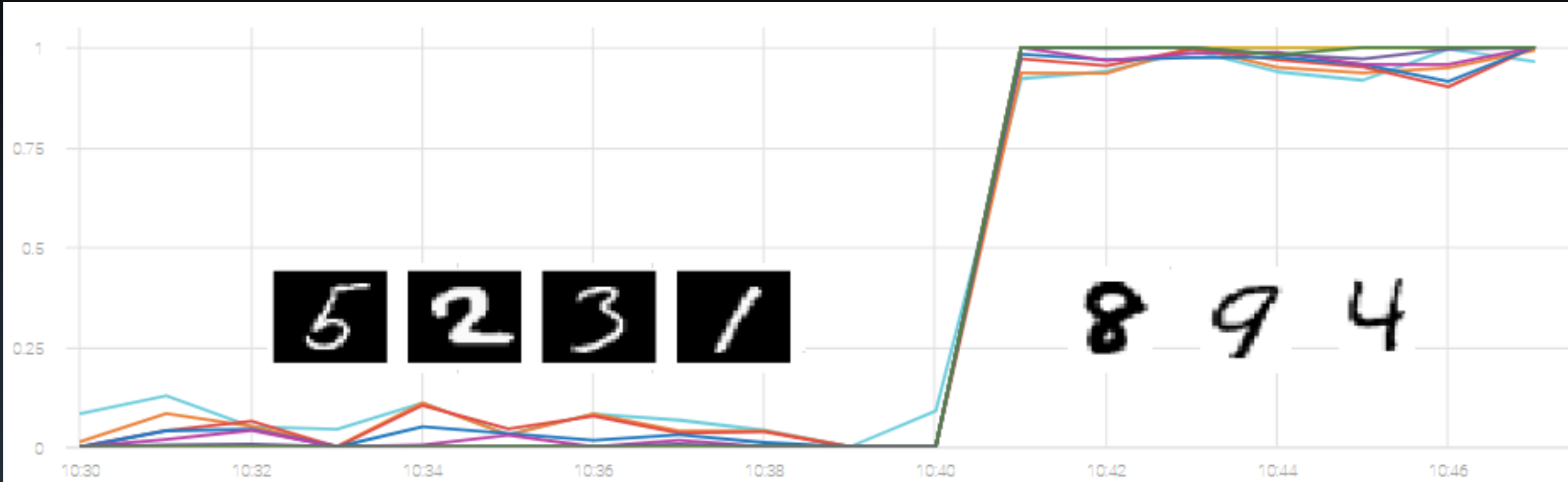
MLWatcher

- range, mean, std, median, q25, q50, q75, iqr for any continuous values (probabilities, features)
- count, frequency for any discrete values (labels, classes)

Python-агент, который позволяет мониторить *статистические* метрики

Sampling

Пример



Метрики улетают когда цвета внезапно инвертируются

Использование

1. Уложить MLWatcher рядом со своим приложением 📁😱
2. Поднять сервер
3. Начать с определённой частотой репортить метрики на сервер

Общие метрики

Мониторить параметры модели - мало

Надо мониторить

1. Системный метрики

1.1. la

1.2. free

1.3. latency...

2. Бизнес-метрики модели (например потерянные клиенты)

Инструменты

- Prometheus
- Grafana
- Zabbix etc

All ▾

predictor

All ▾

version

All ▾

model_name

All ▾

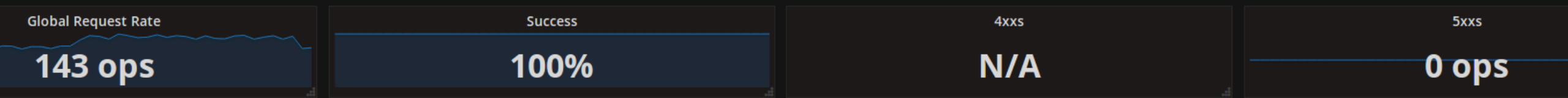
model_image

All ▾

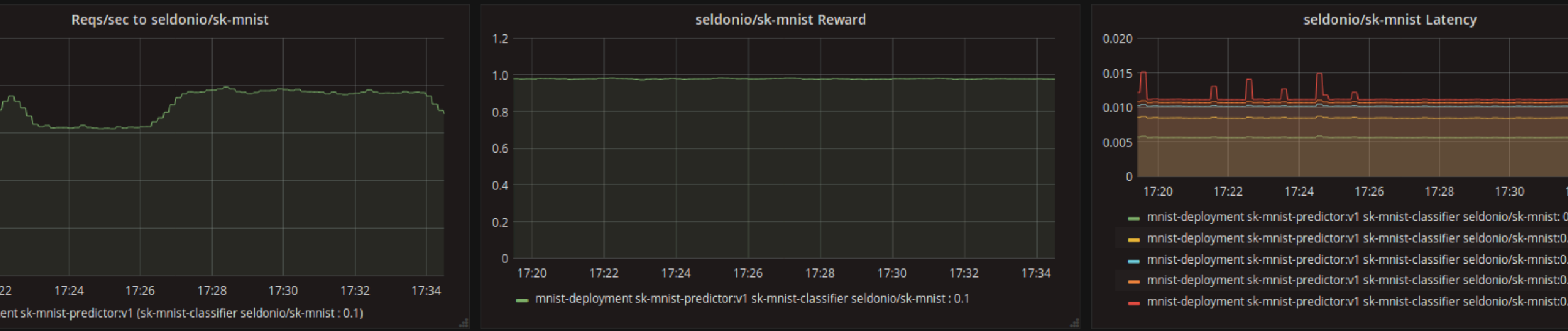
model_version

All ▾

Seldon Core API Dashboard



Models



Мониторинг — важнейшая часть продуктивизированного ML

Без него не имеет смысла вообще продуктивизировать — будет
непредсказуемо

Чему мы научились

1. MLOps — это процессы и командное взаимодействие
2. Чтобы построить MLOps надо иметь базовый набор инструментов
3. Кроме стандартных инструментов есть и непривычные: DVC, MLFlow
4. Продуктивизация ML — это большой путь, который надо пройти

QA

Паша Финкельштейн: if () @asm0di0 else @asm0dey

Михаил Марюфич: @mikhailmar